

块设备、分区表与设备签名

从“这是不是目标盘”到“磁盘、内核与 udev 是否一致”：把高风险存储操作放进一条可证明的证据链。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：任何分区写操作都必须先证明设备身份、已有布局、内容签名和当前使用状态；写入后再证明磁盘表、内核设备树与 udev 已经收敛。

01 识别设备从路径、父子关系和身份字段确认目标对象

02**识别布局**区分 GPT/MBR、分区条目和真实扇区边界

03**识别内容**读取文件系统、Swap、LVM、RAID 等签名

04**决定操作**只在证据门通过后选择 parted 或 fdisk

05**同步视图**让磁盘表、内核视图和 udev 节点重新一致

06**分层验收**分别证明布局、节点、持久标识和内容边界

专题地图

- 知识专题 从磁盘到分区：块设备对象怎样关联
- 知识专题 GPT、MBR 与分区条目分别记录什么
- 知识专题 起点、终点、大小、对齐与分区类型
- 知识专题 设备签名与持久标识
- 操作专题 写入前基线、parted、fdisk 与状态同步
- 诊断专题 新分区未出现、遗留签名与身份冲突
- 经典任务 创建 GPT 分区并完成四层验收
- 经典任务 诊断内核节点缺失和旧签名

阅读时持续回答

- 当前路径到底对应哪一个块设备？
- 分区表、分区类型和内容签名分别属于哪一层？
- 设备是否正在被挂载、Swap、LVM、RAID 或映射层使用？
- 命令修改的是磁盘表、内核视图还是 udev 状态？
- 当前证据能证明什么，又不能证明什么？
- 出现冲突时，下一条最有区分度的证据是什么？

章节边界：本章只把设备、分区表、分区条目、内核视图、持久标识和签名讲清。文件系统进入第 23 章；挂载与 Swap 持久性进入第 24 章；LVM 生命周期进入第 25 章。

第 22 章 · 正文

存储操作最危险的地方，不是命令难记，而是“看起来很像正确设备”的对象太多。`/dev/vdb` 可能是练习盘，也可能已经承载文件系统、Swap、LVM、RAID 或业务数据；`parted` 已经显示新分区，不代表内核立即生成了 `/dev/vdb1`；`FSTYPE` 为空，也不代表设备可以覆盖。一次错误的 `mklabel`、`rm` 或签名清除，可能让原有数据失去正常访问入口。

本章解释块设备、磁盘、分区、分区表、内容签名、内核设备视图和持久标识之间的关系，并沿“身份 → 布局 → 内容 → 占用 → 修改 → 同步 → 验收”推进。最常见的误判，是用一个局部字段替代完整证据：只凭设备名选盘、只凭 `FSTYPE` 判空、只凭分区类型判断内容、只凭命令成功宣布完成。

本章的写操作只改变分区表及分区条目；它不会提前创建文件系统、配置挂载、启用 Swap 或建立 LVM。所有需要真实输出才能确认的版本与设备行为都保留为推荐验证步骤，不声称已经在 RHEL 9 虚拟机实测。

概念

块设备 是内核以固定大小数据块提供随机访问的设备对象，磁盘、分区以及某些映射设备都可能表现为块设备。块设备节点只是访问入口，不自动说明设备的业务用途、是否空闲或是否安全覆盖；观察它应先从 `lsblk` 的父子关系、类型和身份字段开始。

概念

分区 是分区表把整块磁盘的一段地址范围描述成的子对象。分区本身仍是块设备，但它和文件系统、Swap、LVM PV 不是同一对象：分区条目只定义边界和用途元数据，上层内容需要另外创建和验证。

概念

分区表 是写在整块磁盘上的布局元数据，GPT 或 MBR 记录分区条目的起止、类型和标识。重建分区表不是无害初始化，而是替换原布局入口；即使数据区没有全部覆盖，原分区也可能立刻失去正常访问路径。

概念

设备签名 是文件系统、Swap、LVM、RAID 或分区表留在特定偏移的识别元数据。签名只是一段识别入口，却足以影响自动探测和上层工具；删除少量签名字节也可能让完整数据失去正常识别，因此 `wipefs -a` 绝不能成为默认清理步骤。

概念

内核设备视图 是内核当前已经读取并注册的块设备树。磁盘上的分区表已经改变时，内核仍可能暂时保留旧布局；`partprobe` 请求重读分区表，`udevadm settle` 只等待设备事件处理完成，两者解决的是不同层次。

概念

持久标识 把设备名称稳定在某个对象维度，例如硬件或后端身份、控制器路径、分区条目或文件系统内容。`/dev/disk/by-id/`、`by-path/`、`by-partuuid/` 和 `by-uuid/` 的稳定依据不同；它们不是“永不变化”，而是比短设备名更明确地说明自己在跟踪什么。

概念

对齐 是让分区边界与设备逻辑扇区、物理扇区和最优 I/O 单元合理匹配。1 MiB 起点适合许多常见设备，但不是脱离设备拓扑的绝对规则；考试和工作中应优先接受工具的合理默认值，并用扇区、大小和 I/O 字段验证结果。

操作语义速查

以下七个入口分别观察拓扑、签名、分区表、内核重读和 udev 状态。先明确命令作用对象，再记关键形式。

lsblk

SYNOPSIS

```
lsblk [options] [device ...]
```

从 sysfs 与 udev 数据建立当前块设备树，适合确认父子关系、类型、身份字段、分区标识和挂载摘要。

重要参数 / 形式

-o LIST

显式选择输出列；讲义、脚本和验收记录不要依赖默认列。

-f

显示文件系统和 Swap 的常见属性摘要，但不能代替完整签名与占用调查。

-p

使用完整设备路径显示名称。

-J

输出 JSON，便于结构化处理；仍应显式指定列，并在需要父子树时保留 **NAME** 或使用 **--tree**。

blkid

SYNOPSIS

```
blkid [options] [device ...]  
blkid -p [options] device
```

读取 libblkid 可识别的内容标签、类型、UUID 和分区属性；低层探测更适合调查单个可疑设备。

重要参数 / 形式

blkid device

读取常见属性；无普通输出不能直接证明设备为空。

-p

对指定设备进行低层探测，适合冲突或不确定签名。

-o export

以键值形式输出，便于逐项判断。

wipefs

SYNOPSIS

```
wipefs [options] device
```

默认列出可识别签名和偏移；带删除选项时会破坏这些识别入口。

重要参数 / 形式

wipefs device

只读列出签名，是调查流程的默认形式。

-n

与 **-a** 或 **-o** 组合时，以 no-act 方式预览将处理的签名，不执行写入。

-a

删除指定设备上所有可见签名；作用于整盘时通常只清除整盘可见的分区表签名，不会递归清除各分区内容。高风险，禁止作为空盘检查或默认清理步骤。

-o OFFSET

只处理已确认偏移的签名，仍需先完成身份和占用调查。

parted

SYNOPSIS

```
parted [options] [device [command [parameters ...]]]
```

查看或直接修改 GPT/MBR 分区表；许多修改会立即写盘，没有统一的最终提交阶段。

重要参数 / 形式

print

显示当前分区表、分区条目和设备信息。

unit MiB|s

指定输入与显示单位，避免把大小误当终点。

mklabel gpt|msdos

重建整盘分区表，属于极高风险写操作。

mkpart NAME [FS-TYPE] START END

GPT 场景：NAME 是分区名称；FS-TYPE 只用于设置合适的分区类型提示，不会创建文件系统。

mkpart PART-TYPE [FS-TYPE] START END

MBR 场景：PART-TYPE 通常为 primary、extended 或 logical；起止必须使用明确单位。

rm NUMBER

删除分区条目，不迁移数据，也不安全擦除数据区。

fdisk

SYNOPSIS

```
fdisk [options] device
fdisk -l [device ...]
```

查看或交互式编辑分区表；交互修改先保留在内存中的未提交布局里，直到写入命令提交。

重要参数 / 形式

-l

只读列出磁盘、扇区、I/O 大小、分区边界和类型。

p

打印当前未提交布局。

n / t / d

分别创建分区、修改类型和删除条目。

w

把未提交布局写入磁盘并退出。

q

放弃尚未写入的交互修改并退出。

partprobe

SYNOPSIS

```
partprobe [options] [device ...]
```

向内核报告分区表变化，请求重新读取指定磁盘的布局。

重要参数 / 形式

partprobe device

请求重读单个目标磁盘，错误信息是后续占用调查的重要证据。

-s

显示设备及其分区摘要：它仍不能替代 parted、lsblk 的分层验证。

-d

只测试而不通知内核；用于区分探测与实际重读。

udevadm info / settle

SYNOPSIS

```
udevadm info [options] device
udevadm settle [options]
```

info 查询设备属性和 udev 数据库；settle 等待当前设备事件队列处理完成。

重要参数 / 形式

info --query=property --name=device

读取指定设备的属性，适合验证 PARTUUID、分区名称和路径信息。

settle

等待事件收敛；它不修改分区表，也不保证内核重读成功。

monitor

观察后续内核与 udev 事件，适合诊断设备节点没有按预期生成。

安全边界：任何写操作前必须证明设备身份、挂载与 Swap 状态、LVM/RAID/device-mapper 归属以及已有签名。`wipefs -a`、`mklabel` 和删除分区都不能用来“试试看能否清干净”。证据不足时，正确动作是停止写入并继续调查。

[知识专题] 从磁盘到分区：块设备对象怎样关联

刚接触存储时，容易把“设备文件”“磁盘”“分区”“文件系统”混成一个对象。最顺的切入方式是从内核设备树开始：整块磁盘是父对象，分区是它的子对象；文件系统、Swap、LVM 或 RAID 签名可以写在整盘或分区上，但它们属于更高一层的内容。本章只负责识别这些内容，不完整展开文件系统、挂载和 LVM 生命周期。

① [知识点] 磁盘与分区都是块设备，但承担不同角色

lsblk 的 TYPE 字段可以帮助区分常见对象：

TYPE	常见含义	本章判断重点
disk	整块磁盘或虚拟磁盘	可承载分区表，也可能直接承载签名
part	分区表描述的磁盘范围	必须确认所属父磁盘和边界
rom	光驱类设备	通常不作为分区目标
loop	回环设备	多由镜像或容器运行时建立
lvm	device-mapper 上的 LVM 逻辑卷	属于第 25 章主对象
crypt	加密映射设备	不是普通分区条目

TYPE=part 只能证明内核把它识别为分区设备，不能证明其内容类型、是否空闲或是否安全删除。

② [知识点] 设备命名反映驱动和总线，不保证长期不变

常见命名形式包括：

- SCSI、SATA、USB 及许多虚拟控制器：/dev/sda、/dev/sdb；
- virtio-blk：/dev/vda、/dev/vdb；
- NVMe：整盘 /dev/nvme0n1，分区 /dev/nvme0n1p1；
- MMC：整盘 /dev/mmcblk0，分区 /dev/mmcblk0p1。

NVMe 与 MMC 名称中的 `p` 用于把磁盘编号和分区编号分开。设备枚举顺序可能因控制器探测、虚拟机配置或硬件变化而改变，因此不能把“上次是 `/dev/sdb`”当作本次写入依据。

③ [知识点] 父子关系比短设备名更能解释对象

以下字段共同建立拓扑：

- `NAME`：简短内核名称；
- `PATH`：完整设备路径；
- `TYPE`：设备类型；
- `PKNAME`：父内核设备名；
- `MAJ:MIN`：内核主次设备号；
- 树形缩进：显示分区、映射层和父设备关系。

对 `/dev/vdb2` 进行操作前，应确认它确实位于目标 `/dev/vdb` 下，而不是同名脚本变量误指向另一块盘。

④ [知识点] 身份确认需要多条一致证据

破坏性写入前，至少交叉核对以下几类证据：

```
设备路径和父子关系
→ 容量
→ MODEL / SERIAL / WWN（可用时）
→ 现有分区表与签名
→ 挂载点和上层归属
→ 题目或变更单指定的目标
```

虚拟磁盘可能没有 `SERIAL` 或 `WWN`；这不表示无法识别，而是需要依靠虚拟机配置、容量、控制器槽位、父子关系和现有状态组合判断。同容量磁盘较多时，单凭大小仍不足以放行写操作。

⑤ [知识点] `lsblk` 默认输出不是稳定接口

不同 `util-linux` 版本可能调整默认列，因此讲义、考试记录和脚本应显式指定字段。调查命令可以分为两个层次：

```
# 快速拓扑
lsblk -o NAME,PATH,SIZE,TYPE,PKNAME,MOUNTPOINTS

# 破坏性操作前的扩展视图
lsblk -o
NAME,PATH,MAJ:MIN,SIZE,TYPE,PKNAME,FSTYPE,FSVER,LABEL,UUID,PARTLABEL,PARTUUID,PTTYPE,PTUUID,MOUNTPOINTS,MODEL,SERIAL,WWN
```

若某列在目标 RHEL 9 小版本不可用，应通过 `lsblk --help` 查看本机支持的列并移除该列，而不是改用默认输出后猜字段位置。

[Cheatsheet] 先看树，再看身份；`PATH` 是访问入口，`SERIAL/WWN` 是身份线索，`PKNAME` 说明父子关系；任何单个字段都不足以证明设备可以覆盖。

[知识专题] GPT、MBR 与分区条目分别记录什么

分区表是整块磁盘上的元数据结构，它描述分区边界、类型和标识。选择 GPT 或 MBR，不是在现有数据盘上切换显示风格，而是在决定磁盘布局格式。理解这一点后，`mklabel` 的风险、分区类型的边界和持久标识的归属会更清楚。

① [知识点] 分区表属于整盘，分区条目描述地址范围

一个分区条目至少需要表达：

- 分区编号；
- 起始位置；
- 结束位置或长度；
- 类型信息；

- 在 GPT 中可用的分区名称和唯一标识。

分区表写入磁盘后，内核仍需重新读取这些条目，才会生成或更新分区设备节点。

② [知识点] MBR 是受历史结构限制的旧布局

传统 MBR 在磁盘开头保存引导信息和四个分区表项，因此常见“最多四个主分区”的限制。若需要更多分区，可使用一个扩展分区作为容器，再在其中建立逻辑分区。

在 512 字节逻辑扇区的常见情形下，MBR 使用 32 位扇区地址，通常面临约 2 TiB 的寻址限制。该限制与扇区大小有关，因此不要把“2 TiB”脱离上下文解释为所有设备上的绝对数学上限。

③ [知识点] GPT 提供主、备份元数据和 GUID 标识

GPT 通常在磁盘前部保存主头和分区条目数组，并在磁盘末尾保存备份。它还包含保护性 MBR，以降低旧工具把 GPT 磁盘误认为空盘的风险。

GPT 使用 GUID 表达磁盘和分区身份及类型，通常比 MBR 更适合现代大容量磁盘和较多分区。工具常默认建立 128 个分区条目，但这属于常见实现选择，不应误记为 GPT 格式只能有 128 个分区。

④ [知识点] PTYPE、PTUUID、PARTUUID 和 PARTLABEL 不在同一层

标识	所属对象	典型变化时机
PTTYPE	整盘分区表格式	GPT 与 MBR 之间重建布局时改变
PTUUID	整盘分区表标识	重建分区表时通常改变
PARTUUID	某个分区条目	删除并重建分区时通常改变
PARTLABEL	GPT 分区名称	管理员可修改，且不保证全局唯一
文件系统 UUID	分区或设备上的文件系统签名	重新格式化时通常改变，归第 23 章深入

不能用文件系统 UUID 代替分区表 UUID，也不能因为 PARTLABEL=data 就认定它必然承载某个文件系统。

⑤ [知识点] mklabel 是重建布局，不是无害初始化

```
parted -s <DISK> mklabel gpt
```

这条命令会在目标磁盘上建立新的 GPT 元数据，使原分区布局失去正常访问入口。即使数据区未被逐字节覆盖，原内容也可能无法再通过原分区节点访问。因此只有在设备身份、占用和数据保留要求已经明确时才能执行。

[Cheatsheet] GPT/MBR 属于整盘；分区条目属于表；PTUUID 标识表，PARTUUID 标识条目；mklabel 会替换布局，绝不是查看命令。

[知识专题] 起点、终点、大小、对齐与分区类型

分区题经常给出“创建 512 MiB”或“从 1 MiB 开始”的要求。工具实际写入的是扇区边界，因此显示值可能因单位换算和对齐略有差异。稳定做法是明确单位、理解起止语义，并在写入后验证实际结果。

① [知识点] 绝对终点与相对大小是两种表达

parted 常用绝对起止位置，例如：

```
parted -s /dev/vdb mkpart data 1MiB 2049MiB
```

该分区从 1 MiB 到 2049 MiB，名义跨度约 2048 MiB。fdisk 则常在起始扇区处接受相对大小：

```
Last sector ...: +512M
```

考试时应先判断题目给的是最终大小还是终点位置，不能把 2048MiB 直接当作绝对终点。

② [知识点] MiB/GiB 与 MB/GB 不是同一种单位

- MiB、GiB 使用 1024 的幂；
- MB、GB 使用 1000 的幂；
- s 表示扇区。

对大小评分敏感的任务，优先使用题目要求的单位或明确的二进制单位，并在输出中检查实际扇区数和显示大小。

③ [知识点] 对齐是为了匹配设备 I/O 边界

现代分区工具会依据逻辑扇区、物理扇区和最优 I/O 大小选择合适的默认起点。使用 1 MiB 起点通常能与常见 4 KiB 物理扇区和更大擦除块较好对齐，但它不是“所有设备唯一正确”的神奇数字。

可用以下证据检查几何信息：

```
lsblk -o NAME,PHY-SEC,LOG-SEC,MIN-IO,OPT-IO,ALIGNMENT
fdisk -l <DISK>
parted <DISK> unit s print
```

在考试新盘上，接受工具合理默认值通常比手工输入未经计算的扇区更安全。

④ [知识点] 分区类型表达预期用途，不创建上层对象

将分区设为 LVM、Swap 或 Linux filesystem 类型，只改变分区表中的用途元数据。它不会：

- 创建文件系统；
- 写入 Swap 签名；
- 创建 LVM PV；
- 自动挂载或启用设备。

后续章节会分别处理这些上层对象。本章只负责正确设置和验证分区条目。

⑤ [知识点] MBR 类型代码和 GPT 类型不能机械混用

模拟题可能在 MBR 场景中要求把类型代码设为 82 或 8e。GPT 使用 GUID 类型，现代 fdisk 也可能显示类型别名。稳定做法是：

```
先确认当前分区表类型
→ 在工具中列出可选类型
→ 选择与题意匹配的名称或代码
→ 写入后重新打印验证
```

不要把某个 MBR 十六进制代码背成所有环境下的唯一输入。

⑥ [知识点] parted 的文件系统类型提示不等于 mkfs

某些 parted mkpart 形式允许提供 FS-TYPE，该信息主要帮助选择分区类型或兼容标志。它不会在分区上建立文件系统。创建完成后，如果 FSTYPE 仍为空，这是符合本章任务边界的状态。

[Cheatsheet] parted 常用绝对起止，fdisk 常用 +SIZE；单位写清楚；类型是元数据，不是格式化；边界以写入后的扇区和大小为准。

[知识专题] 设备签名与持久标识：设备上已经有什么

分区表只描述布局，内容签名告诉工具“这个地址范围看起来是什么”。文件系统超级块、Swap 头、LVM 标签和 RAID 元数据都可能被 libblkid 识别。旧实验留下的签名可能与新分区表同时存在，导致自动探测、安装程序或管理工具给出冲突判断。

① [知识点] 签名是识别入口，不是完整数据本身

签名通常是位于特定偏移的魔数和元数据片段。工具通过它识别类型、版本、标签和 UUID。删除签名可能只改动少量字节，却足以使正常工具无法识别原内容，因此仍然属于破坏性操作。

② [知识点] 一个设备可以残留多层或多种签名

例如，一个曾经作为 LVM PV 的分区后来被直接格式化，设备上可能同时残留旧 LVM 元数据和新文件系统签名。又如，整盘曾使用 GPT，后来创建 MBR 时，磁盘末尾仍可能残留 GPT 备份头。

看到多个签名时，不能立即把它们全部视为垃圾。需要判断：

```
哪个签名对应当前有效对象
→ 哪个签名是历史残留
→ 上层是否仍在使用
→ 题目是否明确允许丢弃
```

③ [知识点] blkid 普通模式与低层探测用途不同

常见查询：

```
blkid <DEVICE>
blkid -p <DEVICE>
blkid -p -o export <DEVICE>
```

普通模式适合读取常见标签、UUID 和类型；低层 `-p` 直接探测指定设备，更适合调查冲突或不确定签名。若普通 `blkid` 没有输出，不能直接得出“设备为空”的结论。

④ [知识点] wipefs 默认列出签名和偏移

```
wipefs <DEVICE>
```

该命令用于列出工具可识别的签名及其偏移，不会因为命令名包含“wipe”就默认清除内容。真正危险的是带删除选项的形式，例如 `-a`。调查流程中应保持只读。

⑤ [知识点] wipefs -a 不是空盘检查命令

`wipefs -a <DEVICE>` 会删除指定设备上 libblkid 可见的签名。对整盘设备执行时，通常只清除整盘上的分区表签名，不会递归清除各分区内部的文件系统或 LVM 签名；对分区设备执行时，则可能清除该分区上的文件系统、Swap、LVM 或 RAID 签名。它不等于安全擦除数据，但足以破坏正常识别入口，因此默认答案中禁止无调查使用。

若题目明确授权丢弃设备内容，仍应先：

1. 保存 `wipefs` 和 `blkid -p` 输出；
2. 确认设备未被挂载、启用或作为上层成员；
3. 使用 `wipefs -n -a <DEVICE>` 进行模拟检查；
4. 优先定向清除已确认的签名，而不是盲目全清；
5. 清除后重新验证分区表、签名和设备视图。

⑥ [知识点] 文件系统、Swap、LVM、RAID 签名需要不同的上层证据

`FSTYPE=xfs`、`TYPE=swap`、`LVM2_member` 或 `linux_raid_member` 只是入口。真正决定是否可覆盖，还要查看对应上层对象是否存在和是否活动。例如：

- 文件系统：检查挂载关系；
- Swap：检查当前是否启用；
- LVM：检查是否属于 PV/VG/LV；
- RAID：检查是否属于阵列；
- device-mapper：检查是否有映射和 holders。

本章只建立调查接口，不完整展开这些上层生命周期。

⑦ [知识点] 持久链接稳定在不同维度

链接目录	主要稳定维度	可能变化的场景
/dev/disk/by-id/	设备序列、WWN 或后端标识	后端克隆、序列缺失或虚拟化配置变化
/dev/disk/by-path/	控制器、端口和拓扑路径	设备换端口或控制器路径改变
/dev/disk/by-partuuid/	GPT/MBR 分区条目标识	删除重建分区或重建分区表
/dev/disk/by-partlabel/	GPT 分区名称	改名或出现同名分区
/dev/disk/by-uuid/	文件系统或 Swap UUID	重新格式化或重写签名，归后续章节深入

持久链接不是“永不变化”，而是把名称稳定在某个对象维度。选择前先明确自己要稳定的是硬件身份、物理路径、分区条目还是文件系统内容。

[Cheatsheet] `blkid` 读常见属性，`blkid -p` 做低层探测，`wipefs` 看签名和偏移；无输出不等于空盘，多签名不等于都可删除。

[操作专题] 破坏性操作前建立设备基线

稳定的存储操作从调查开始，而不是从 `fdisk` 或 `parted` 开始。最顺的顺序是：先确认目标身份、再确认布局和签名，随后确认当前占用和上层归属，最后决定是否允许写入。

① [查询] 先建立拓扑和身份视图

```
lsblk -o NAME,PATH,MAJ:MIN,SIZE,TYPE,PKNAME,MOUNTPOINTS,MODEL,SERIAL,WWN
```

作用对象：内核当前维护的块设备树。验证重点：目标路径、父子关系、容量和身份字段是否与题意一致。边界：该命令不读取所有历史签名，也不能证明设备没有被其他上层对象使用。

② [查询] 查看分区表和分区边界

```
parted <DISK> unit MiB print
fdisk -l <DISK>
```

`parted` 适合明确显示分区表类型、起止和名称；`fdisk -l` 提供扇区、I/O 大小和分区类型的独立视角。两条输出应指向同一设备，并与 `lsblk` 的父子关系一致。

③ [查询] 探测内容签名

```
blkid -p <DISK>
wipefs <DISK>

# 对已有分区逐一检查
blkid -p <PARTITION>
wipefs <PARTITION>
```

对整盘和分区分别检查，因为分区表签名位于整盘，而文件系统、LVM 或 RAID 签名可能位于分区中。

④ [查询] 检查挂载、Swap 和上层归属

```
lsblk -o NAME,PATH,TYPE,FSTYPE,MOUNTPOINTS
findmnt --source <DEVICE>
swapon --show
pvs
cat /proc/mdstat
```

这些命令分别排除常见占用。若设备存在 device-mapper 映射、加密层或 holders，还应检查：

```
ls -l /sys/class/block/<KERNEL_NAME>/holders/
```

不存在某一种占用，不等于不存在其他占用。

⑤ [验证点] 明确写入放行条件

只有以下条件同时满足时，才进入创建或删除分区：

- 设备身份与目标一致
- 现有布局和签名已记录
- 没有需要保留的数据
- 没有当前挂载、Swap 或上层成员关系
- 题目或变更单明确授权改变该设备

若任何条件不明确，正确动作是停止并继续调查，而不是使用 `--force`、`wipefs -a` 或重新格式化来“清理异常”。

⑥ [工作迁移] 保存变更前基线

真实工作中可把输出保存到变更记录：

```
stamp=$(date +%Y%m%d-%H%M%S)
lsblk -0 > "lsblk-before-${stamp}.txt"
parted <DISK> unit s print > "parted-before-${stamp}.txt"
fdisk -l <DISK> > "fdisk-before-${stamp}.txt"
wipefs <DISK> > "wipefs-before-${stamp}.txt"
```

考试中不必机械创建文件，但应保留“先观察、后写入、再比较”的思路。

[Cheatsheet] 身份 → 布局 → 签名 → 占用 → 授权；任一证据冲突都停止。FSTYPE 为空只是一个字段，不是写入许可。

[操作专题] 使用 parted 查看和修改分区表

parted 同时支持 GPT 和 MBR，并适合使用明确单位进行非交互操作。它的危险点是许多修改会立即写入磁盘，不存在统一的“最后按 w 才提交”阶段。因此每条命令前都应重新确认设备路径。

① [查询] 查看当前布局和帮助

```
parted <DISK> print
parted <DISK> unit MiB print
parted <DISK> help
```

在交互模式中可以先执行 `print`，确认设备型号、容量、分区表类型和现有条目。`unit` 只改变显示和输入单位，不改变已有分区内容。

② [操作] 在确认可丢弃的新盘上建立 GPT

```
parted -s <DISK> mklabel gpt
```

作用对象：整块磁盘的分区表。风险：原布局失去正常访问入口。验证：立即运行 `parted <DISK> print` 和 `lsblk`，确认没有写错设备。

③ [操作] 使用明确起止创建分区

```
parted -s <DISK> mkpart data 1MiB 2049MiB
parted -s <DISK> mkpart future-lvm 2049MiB 3073MiB
```

GPT 的第一个参数常作为分区名称。这里没有创建文件系统或 LVM PV，只建立了两个分区条目。

④ [参数] 设置分区用途标志

```
parted -s <DISK> set 2 lvm on
```

该操作把第 2 分区标记为 LVM 用途。它不验证分区中是否已有 LVM 签名，也不会执行 `pvcreate`。

⑤ [操作] 删除分区条目

```
parted -s <DISK> rm <NUMBER>
```

`rm` 删除分区表中的条目，不会安全迁移或擦除原数据区。删除后，上层对象可能立即失去设备节点。执行前必须确认编号、父磁盘、挂载和上层归属。

⑥ [验证点] 同时检查磁盘表和内核树

```
parted <DISK> unit MiB print
fdisk -l <DISK>
lsblk -o NAME,PATH,START,SIZE,TYPE,PARTLABEL,PARTUUID,PARTTYPE <DISK>
```

`parted` / `fdisk` 读取磁盘布局，`lsblk` 主要反映内核与 `udev` 视图。若两者不一致，进入同步诊断，不要重复执行 `mklabel` 或 `mkpart`。

⑦ [帮助入口] 不确定子命令时先查看本机帮助

```
parted --help
parted <DISK> help mkpart
man parted
```

版本差异最容易出现在类型、`flag` 和脚本化输出上。考试环境以本机帮助为准。

[Cheatsheet] `print` 先看；`mklabel` 改整盘；`mkpart` 改条目；`set ... lvm on` 只改用途；`parted` 修改常立即生效。

[操作专题] 使用 `fdisk` 交互式管理分区

`fdisk` 的特点是交互式未提交布局：多数修改先保存在内存中，执行 `w` 才写入磁盘，执行 `q` 可放弃未写入变化。这并不降低设备选错的风险，但提供了提交前再次打印检查的机会。

① [查询] 只读列出布局

```
fdisk -l <DISK>
```

输出可包含逻辑/物理扇区大小、I/O 大小、磁盘标签类型、磁盘标识和分区表。人类可读输出可能随版本变化，不应依赖固定列宽进行脆弱解析。

② [操作] 进入交互并先打印

```
fdisk <DISK>
```

进入后首先使用：

```
p    打印当前未提交布局
m    查看帮助
```

当前未提交布局可能已包含本轮尚未写入的变化，因此 **p** 显示的是当前交互会话状态，不一定等于磁盘原始状态。

③ [操作] 创建分区

常见按键：

n	新建分区
<Enter>	接受合理默认起始扇区
+512M	指定相对大小

默认起点通常已经考虑对齐。若题目没有强制扇区，不要为了“看起来整齐”输入未经计算的起点。

④ [操作] 修改分区类型

t	修改类型
L	列出类型（提示中支持时）

先确认当前是 GPT 还是 MBR，再选择类型名称、别名或代码。写入前用 **p** 检查最终类型。

⑤ [操作] 删除未提交的分区条目

d	删除分区条目
----------	--------

此时通常还未写盘，但删除后继续执行 **w** 就会提交。若发现选错设备或编号，使用 **q** 退出，不要通过继续创建来“补回来”。

⑥ [提交边界] **w** 与 **q** 的意义

w	写入分区表并退出
q	不保存本次未写入变化并退出

q 只能放弃当前交互会话中尚未写盘的修改，不能撤销之前已执行并写入的其他工具命令。

⑦ [验证点] 写盘后重新从外部读取

退出后执行：

```
fdisk -l <DISK>
parted <DISK> unit MiB print
lsblk -o NAME,PATH,START,SIZE,TYPE,PARTTYPE,PARTUUID <DISK>
```

不要只相信交互会话最后一次 **p** 的屏幕内容。

[Cheatsheet] **p** 看未提交布局，**n** 新建，**t** 改类型，**d** 删除，**w** 写盘，**q** 放弃未提交；写盘后必须从外部重新查询。

[操作专题] 让磁盘布局、内核视图和 udev 重新一致

分区工具已经显示新条目，但 `/dev/vdb2` 没有出现，说明故障不一定在分区表。需要分开判断：磁盘上是否已写入、内核是否重读、udev 是否完成节点和链接处理。

① [知识点] 三个状态源不能互相替代

磁盘上的分区表
→ 内核块设备和分区对象
→ udev 设备节点、属性与持久链接

`parted print` 能看到新分区，只证明磁盘表可被工具读取；`lsblk` 能看到分区，说明内核已建立对象；`/dev/disk/by-partuuid/` 出现链接，说明相应 `udev` 规则已完成。

② [操作] 请求内核重读分区表

```
partprobe <DISK>
```

`partprobe` 通知操作系统分区表已改变。它不会创建新的分区表，也不会清除签名。如果设备正在使用，内核可能拒绝安全更新。

③ [操作] 等待 udev 事件处理

```
udevadm settle
```

该命令等待当前设备事件队列处理完成。它不会主动重读分区表，也不会修复错误的分区条目。

④ [查询] 查询具体设备的 udev 属性

```
udevadm info --query=property --name=<PARTITION>
```

可核对 `ID_PART_ENTRY_UUID`、`ID_PART_ENTRY_NAME`、父设备和其他属性。实际字段取决于设备和规则。

⑤ [验证点] 检查设备节点与持久链接

```
lsblk -o NAME,PATH,TYPE,PKNAME,PARTLABEL,PARTUUID <DISK>
ls -l /dev/disk/by-partuuid/
ls -l /dev/disk/by-partlabel/ 2>/dev/null
```

链接目录可能不存在或没有匹配条目，尤其在无标签、非 GPT 或设备属性不足时。不能把某一链接缺失直接解释为分区表损坏。

⑥ [边界] 设备繁忙时不要重复写分区表

若 `partprobe` 报告设备忙或无法更新，下一步应调查挂载、Swap、LVM、RAID、device-mapper 和打开使用者。重复执行 `mklabel`、`mkpart` 或重启分区工具，只会增加数据风险。

⑦ [边界] 重启是运行决策，不是默认修复

在生产环境中，若内核无法安全在线重读而变更又必须生效，可能需要维护窗口和重启。但在考试新盘场景，优先确认设备未使用并完成 `partprobe` / `udev` 验证。不能把重启作为跳过调查的通用答案。

[Cheatsheet] `parted` 看磁盘表，`partprobe` 请求内核重读，`udevadm settle` 等事件，`lsblk` 和 `/dev/disk/` 验收；三层顺序不能倒置。

[诊断专题] 新分区未出现或旧布局仍可见

最常见症状是：分区工具显示已经创建，但 `lsblk` 没有对应节点；或者删除分区后，旧节点仍短时间存在。诊断必须先确定“哪一层不同步”，而不是立即重新写盘。

① [诊断点] 先证明分区表是否真的写入目标磁盘

```
parted <DISK> unit s print
fdisk -l <DISK>
```

若两者都没有新条目，问题可能是命令未执行、选错设备、交互修改未 `w` 提交，或命令报错。此时不应先运行 `partprobe`，因为磁盘上没有可同步的新布局。

② [诊断点] 比较磁盘表与内核树

```
lsblk -o NAME,PATH,START,SIZE,TYPE,PKNAME <DISK>
```

磁盘表有新条目而 `lsblk` 没有，是典型的内核视图滞后。下一条最有区分度的证据是 `partprobe` 的返回和内核日志，而不是再次创建同一分区。

③ [诊断点] 执行最小同步

```
partprobe <DISK>
udevadm settle
```

随后重新运行 `lsblk`。如果节点出现，继续验证 `PARTUUID` 和持久链接；如果仍未出现，保留错误信息并进入占用调查。

④ [诊断点] 调查设备是否正在使用

```
findmnt
swapon --show
pvs
cat /proc/mdstat
ls -l /sys/class/block/<KERNEL_NAME>/holders/
```

设备使用状态可能使内核无法在线接受新的布局。此时最小修复不是强制删除，而是让上层对象安全释放，或将变更安排到维护窗口。

⑤ [诊断点] 查看内核设备事件证据

```
journalctl -k --since "-10 min"
udevadm monitor --kernel --udev --property
```

`udevadm monitor` 适合在再次触发设备事件时观察内核与 `udev` 流程，但不应为了产生事件而重复破坏性写入。

⑥ [再验证] 四层证据必须收敛

```
parted/fdisk: 磁盘表正确
lsblk: 内核节点正确
udevadm / /dev/disk: 属性和链接正确
blkid/wipefs: 没有非预期内容变化
```

[Cheatsheet] 有表无节点：先 `partprobe`；仍失败：查使用者和内核日志；绝不通过重复 `mklabel` 解决同步问题。

[诊断专题] 未知、冲突和遗留设备签名

另一个高频症状是：安装程序、LVM 或文件系统工具提示设备已有内容，普通 `blkid` 却没有清晰输出；或者同一分区同时显示多个历史类型。诊断目标不是尽快“擦干净”，而是确认哪些签名仍代表有效对象。

① [诊断点] 先使用低层探测和签名列表

```
blkid -p -o export <DEVICE>
wipefs <DEVICE>
```

记录识别类型、UUID、标签和偏移。不同工具显示的信息可能互补，不应只取一条输出下结论。

② [诊断点] 判断签名所属层

- GPT/MBR 签名通常属于整盘分区表；

- 文件系统、Swap、LVM PV 或 RAID 成员签名常位于整盘或分区；
- device-mapper 或 LVM 逻辑卷是内核映射对象，不能仅靠底层签名解释其当前状态。

③ [诊断点] 关联上层当前状态

```
findmnt --source <DEVICE>
swapon --show
pvs -o pv_name,vg_name,pv_uuid
cat /proc/mdstat
lsblk -o NAME,PATH,TYPE,FSTYPE,MOUNTPOINTS
```

若签名对应当前活动对象，停止清除。若题目声明设备为可丢弃练习盘，也必须确保没有误选父盘或相邻分区。

④ [诊断点] 普通 `blkid` 无输出的解释边界

可能原因包括：

- 没有已知签名；
- 权限或缓存行为不同；
- 多个冲突签名造成歧义；
- 签名损坏或版本不识别；
- 探测对象选择错误。

因此下一步应是 `blkid -p` 与 `wipefs`，不是直接格式化。

⑤ [最小修复] 只有授权后才设计清除

先模拟：

```
wipefs -n -a <DEVICE>
```

如果确认仅某个历史签名需要移除，可依据 `wipefs` 显示的偏移进行定向处理，并在执行前保存输出。实际清除命令必须由任务明确授权，不应作为本章默认答案。

⑥ [再验证] 清除后重新建立完整证据

```
wipefs <DEVICE>
blkid -p <DEVICE>
parted <DISK> print
lsblk -o NAME,PATH,TYPE,FSTYPE,PARTUUID,MOUNTPOINTS
```

清除签名后，必须确认没有误删当前分区表或有效内容入口。

[Cheatsheet] 多签名先归属，后处理；`blkid` 无输出不等于空；`wipefs -a` 不是诊断动作，任何清除都要有授权、基线和再验证。

[诊断专题] 身份、类型和证据冲突时怎样停止

破坏性操作真正需要训练的是“何时不执行”。以下冲突都应触发停止条件。

① [边界] 同容量不等于同一设备

两块 20 GiB 虚拟磁盘可能只有控制器槽位、序列或现有布局不同。若题目只说“新增磁盘”，先用虚拟机或硬件清单确认新增对象，再将其与 `lsblk` 属性匹配。

② [边界] 分区类型和内容签名不一致并不自动表示故障

`PARTTYPE` 表示分区表中的用途类型，`FSTYPE` 表示探测到的内容签名。以下状态都可能出现：

LVM 分区类型 + 无 LVM 签名：只完成了分区类型设置
Linux filesystem 类型 + XFS 签名：常见有效组合
Swap 类型 + 旧 XFS 签名：可能是复用不完整，需要调查

不能通过修改类型把 XFS “转换”为 Swap，也不能通过清除 FSTYPE 解决所有不一致。

③ [边界] 工具输出冲突时先确认读取层

- `parted / fdisk` 主要看磁盘分区表；
- `lsblk` 主要看内核设备树并融合 `udev`/签名属性；
- `blkid / wipefs` 看内容签名；
- `/dev/disk` 看 `udev` 生成的链接。

先解释每个工具在读什么，再判断是否真的矛盾。

④ [边界] 命令成功不等于目标终态正确

`parted` 返回成功只能说明命令被接受；仍需证明：

- 写到了正确设备；
- 分区边界和类型符合要求；
- 内核节点已更新；
- 没有破坏非目标签名；
- 没有越过文件系统、挂载或 LVM 的章节边界。

⑤ [工作迁移] 变更单应记录身份和回退限制

存储布局变化通常不能像文本配置一样简单回滚。真实工作中至少记录：

变更前设备身份与布局
变更目标和授权范围
预期分区表、边界和类型
内核重读是否要求维护窗口
验证命令和结果
失败时的停止条件

如果没有可靠备份或恢复方案，不应把“重新创建原分区”描述成保证可用的回滚。

[Cheatsheet] 证据冲突时停止；类型不是内容；命令成功只证明局部；存储变更的回退能力必须事先评估。

[经典任务] 在确认空闲的练习盘上创建 GPT 分区

服务器新增一块题目指定为 `/dev/vdb` 的 8 GiB 练习盘。该设备过去可能被其他实验使用，因此不能假设它为空。要求在确认没有需要保留的数据和当前使用者后完成以下终态：

1. 使用 GPT 分区表；
2. 创建名为 `data` 的第 1 分区，从 1 MiB 开始，容量约 2 GiB；
3. 创建名为 `future-lvm` 的第 2 分区，容量约 1 GiB，并设置为 LVM 用途类型；
4. 让内核和 `udev` 识别两个分区；
5. 验证两个分区的 `PARTUUID`；
6. 不创建文件系统，不执行 `pvcreate`，不挂载；
7. 不使用无调查的 `wipefs -a`、`--force` 或重启代替验证。

验收矩阵

层	必须提供的证据
身份	<code>/dev/vdb</code> 的路径、容量、父子关系和可用身份字段
现状	写入前分区表、签名和占用调查
磁盘表	GPT、两个分区的起止、名称和类型
内核	<code>lsblk</code> 中出现 <code>/dev/vdb1</code> 与 <code>/dev/vdb2</code>
udev	<code>PARTUUID</code> 和可用持久链接
内容边界	没有非预期文件系统或 LVM 签名

请先独立完成。参考解答从下一页开始。

[参考解答] 从设备身份到四层验证

① [调查] 建立写入前基线

```
lsblk -o NAME,PATH,MAJ:MIN,SIZE,TYPE,PKNAME,FSTYPE,PARTLABEL,PARTUUID,PTTYPE,PTUUID,MOUNTPOINTS,MODEL,SERIAL,WMN
parted /dev/vdb unit MiB print
fdisk -l /dev/vdb
blkid -p /dev/vdb
wipefs /dev/vdb
findmnt
swapon --show
pvs
cat /proc/mdstat
```

根据真实输出确认：

- `/dev/vdb` 与题目指定的新盘一致；
- 不含需要保留的分区或签名；
- 没有挂载、Swap、LVM 或 RAID 归属；
- 若存在历史签名，题目确实授权丢弃，且已记录具体类型和偏移。

若任何证据不明确，应停止，不执行后续命令。

② [操作] 创建 GPT 和两个分区

```
parted -s /dev/vdb mklabel gpt
parted -s /dev/vdb mkpart data 1MiB 2049MiB
parted -s /dev/vdb mkpart future-lvm 2049MiB 3073MiB
parted -s /dev/vdb set 2 lvm on
```

`mklabel` 只在调查确认后执行。`mkpart` 建立分区条目；`set 2 lvm on` 设置用途类型，不创建 PV。

③ [同步] 请求内核重读并等待 udev

```
partprobe /dev/vdb
udevadm settle
```

若 `partprobe` 报告设备忙或无法重读，不重复写盘；返回占用调查，保存错误信息，并说明需要释放使用者或安排维护窗口。

④ [验证] 分层检查目标终态

```
parted /dev/vdb unit MiB print
fdisk -l /dev/vdb
lsblk -o NAME,PATH,START,SIZE,TYPE,PKNAME,FSTYPE,PARTLABEL,PARTUUID,PARTTYPE,PTTYPE,PTUUID /dev/vdb
udevadm info --query=property --name=/dev/vdb1
udevadm info --query=property --name=/dev/vdb2
blkid -p /dev/vdb1
blkid -p /dev/vdb2
wipefs /dev/vdb1
wipefs /dev/vdb2
```

验收时允许工具因扇区对齐显示略有差异，但应确认：

- 分区表为 GPT；
- 第 1 分区名为 `data`，大小约 2 GiB；
- 第 2 分区名为 `future-lvm`，大小约 1 GiB，类型为 LVM 用途；
- 两个内核节点都存在并具有 `PARTUUID`；
- 没有执行 `mkfs` 或 `pvccreate`，因此内容签名应符合这一边界。

⑤ [典型错误]

- 仅凭 `/dev/vdb` 名称直接执行 `mklabel`；
- 把 `2048MiB` 当作从 1 MiB 开始的绝对终点，得到约 2047 MiB；
- 认为 `set 2 lvm on` 已经创建 PV；
- `parted print` 有分区而 `lsblk` 无节点时重复创建；
- 为清理历史状态默认执行 `wipefs -a`；
- 编造示例 `PARTUUID` 作为验收结果。

[Cheatsheet] 基线调查 → `mklabel gpt` → 两次 `mkpart` → 设置第 2 分区用途 → `partprobe` → `udevadm settle` → 分区表、内核、udev、签名四层验收。

[经典任务] 诊断新分区未出现和遗留签名

管理员报告：已经在 `/dev/vdc` 上创建新分区，`parted /dev/vdc print` 能看到条目，但 `lsblk` 没有显示新的分区节点。该磁盘以前用于实验，普通 `blkid /dev/vdc1` 没有明确输出。当前不允许重新创建分区表，不允许格式化，也不允许清除未经确认的签名。

要求：

1. 证明磁盘上的分区表是否正确；
2. 判断内核是否仍使用旧布局；
3. 请求安全重读并等待 udev；
4. 若设备正在使用，停止并报告具体阻断层；
5. 调查可能的旧文件系统、LVM 或 RAID 签名；
6. 给出最小修复和分层再验证；
7. 不把重启或 `wipefs -a` 作为默认答案。

[参考解答] 用最有区分度的证据推进

① [当前证据] 比较磁盘表与内核视图

```
parted /dev/vdc unit s print
fdisk -l /dev/vdc
lsblk -o NAME,PATH,START,SIZE,TYPE,PKNAME,FSTYPE,MOUNTPOINTS /dev/vdc
```

- 若 `parted` 与 `fdisk` 都没有新条目，先检查交互式 `fdisk` 是否忘记 `w`、命令是否写错设备或执行失败；
- 若磁盘表有条目而 `lsblk` 无节点，假设转为“内核尚未采用新布局”。

② [最小修复] 重读并等待事件

```
partprobe /dev/vdc
udevadm settle
lsblk -o NAME,PATH,START,SIZE,TYPE,PKNAME,PARTUUID /dev/vdc
```

若节点出现，继续签名调查。若仍失败，保存 `partprobe` 的错误并调查占用。

③ [占用调查] 找到阻断层

```
findmnt
swapon --show
pvs
cat /proc/mdstat
ls -l /sys/class/block/vdc/holders/
journalctl -k --since "-10 min"
```

发现活动使用者时，不强制重读。报告是挂载、Swap、LVM、RAID、映射层还是其他内核使用者阻止更新，并提出安全释放或维护窗口方案。

④ [签名调查] 解释普通 `blkid` 无输出

```
blkid -p -o export /dev/vdc1
wipefs /dev/vdc1
```

根据实际输出判断是否存在单一签名、冲突签名或无法识别的内容。再用对应上层查询判断它是否仍有效。未获得授权时，到此为止，不执行清除。

⑤ [再验证] 证明各层收敛

```
parted /dev/vdc unit s print
lsblk -o NAME,PATH,START,SIZE,TYPE,PKNAME,FSTYPE,PARTUUID /dev/vdc
udevadm info --query=property --name=/dev/vdc1
blkid -p /dev/vdc1
wipefs /dev/vdc1
```

最终报告应区分：磁盘表正确、内核节点已更新、udev 属性已生成、签名仍保留或已按授权最小处理。任何一层未完成，都不能扩大为“问题已解决”。

[Cheatsheet] 表有节点无 → `partprobe`；等待事件 → `udevadm settle`；仍失败 → 查使用者和内核日志；签名不明 → `blkid -p` + `wipefs`；未授权不清除。

[本章收束] 每次写入只改变一个存储层

块设备名称、分区表、内核设备树、udev 链接和内容签名各自回答不同问题。稳定的工作方法不是“先清空再重做”，而是始终沿同一条证据链推进：

```
确认设备身份
→ 记录现有布局、签名和占用
→ 只修改分区表或指定条目
→ 请求内核重读并等待 udev
→ 用磁盘表、内核、持久标识和签名分层验收
```

① [操作决策] 写操作前先通过五道证据门

- 目标设备的路径、容量、父子关系和可用身份字段相互一致；
- 现有分区表、分区边界和签名已经记录；
- 挂载、Swap、LVM、RAID、device-mapper 与 holders 已经调查；
- 题目或变更单明确允许改变该设备；
- 失败时的停止条件、维护窗口和恢复限制已经清楚。

任何一道证据门未通过，都不进入 `mklabel`、删除分区或签名清除。

② [检查清单] 完成分区操作后逐层收束

- `parted` 或 `fdisk` 看到预期分区表和边界；
- `lsblk` 看到正确父子关系和内核分区节点；
- `PARTUUID`、`PARTLABEL` 或所需持久链接已经生成；
- `blkid -p` 与 `wipefs` 没有暴露非预期内容；
- 没有越过题目要求创建文件系统、启用 Swap、创建 PV 或挂载；
- 所有结论都说明“能证明什么”和“不能证明什么”。

③ [主要判断表] 证据入口与证明边界

证据入口	能证明什么	不能单独证明什么
<code>lsblk</code>	当前内核块设备树、父子关系、常见属性和部分签名摘要	设备绝对空闲、所有历史签名都不存在
<code>parted ... print</code>	磁盘上的分区表类型、条目、起止和名称	内核已经采用最新布局
<code>fdisk -l</code>	扇区、I/O 大小、分区边界和类型的独立视角	<code>udev</code> 节点已经生成
<code>blkid -p</code>	指定设备当前可探测的低层内容签名	该内容是否仍被上层活动对象使用
<code>wipefs</code>	可识别签名及其偏移	所有数据都已清除，或所有签名都可以删除
<code>partprobe</code>	已向内核请求重读分区表	重读一定成功、 <code>udev</code> 一定已经处理完成
<code>udevadm settle</code>	已等待当前 <code>udev</code> 事件队列收敛	磁盘分区表正确，或设备未被占用

④ [安全边界] 三个最常见的错误扩大

不要把 `FSTYPE` 为空扩大为“设备是空盘”；不要把分区类型扩大为“文件系统、Swap 或 LVM 已经创建”；不要把命令返回成功扩大为“磁盘表、内核、`udev` 与内容层已经全部正确”。存储章节的可靠结论必须来自多层证据收敛。

⑤ [向下一章交接] 从分区设备进入文件系统对象

本章停在“块设备和分区条目已经正确存在，并且身份与签名状态清楚”。第 23 章《文件系统、标签、UUID 与容量管理》将在这个可靠块设备之上创建和识别文件系统，讨论文件系统标签、UUID、容量证据和增长边界；挂载与 Swap 持久性留给第 24 章，LVM 对象生命周期留给第 25 章。

本章最终安全边界只有一句：设备身份和已有内容没有被证据证明之前，不执行破坏性写入。